

# WhatNext Vision Motors: Shaping the Future of Mobility with Innovation and Excellence

## Use Case :

WhatNext Vision Motors, a pioneering force in the automotive industry, is dedicated to transforming the mobility sector with innovative technology and solutions that prioritize customer needs. The company has embarked on an ambitious Salesforce project with the core objective of enhancing the customer experience and streamlining its operational processes.

At the heart of this project is the improvement of the customer ordering process. The system is designed to automatically suggest the nearest dealer location to customers based on their address. This feature is intended to significantly enhance the convenience and efficiency of the ordering experience, making it more customer-friendly and reducing the time and effort required from the customer's end.

The project also addresses a common issue in the automotive industry: stock availability. The system includes a mechanism that prevents customers from placing orders for vehicles that are out of stock. This proactive approach ensures that customers can only create orders for vehicles that are currently available, thus avoiding potential confusion and disappointment that may arise from stock unavailability. This feature not only enhances customer satisfaction but also improves the accuracy of the company's order fulfillment process.

Furthermore, the project incorporates a scheduled process for updating the status of bulk order records. This automated process is designed to update the order status based on stock availability. If a vehicle is out of stock at the time of order placement, the system will update the order status to 'Pending.' On the other hand, if the vehicle is in stock, the system will update the status to 'Confirmed.' This ensures that all orders are accurately reflected in terms of their fulfillment status, providing clear and transparent communication to customers regarding the status of their orders.

The implementation of this Salesforce project at WhatNext Vision Motors is expected to yield several benefits. It aims to create a more efficient ordering system that reduces the potential for errors and improves the overall service provided to customers. By streamlining the ordering process and ensuring accurate stock availability, the company can enhance customer satisfaction and loyalty.

Moreover, the project is expected to contribute to operational efficiency by reducing the administrative burden on staff. By automating certain processes, employees can focus on more strategic tasks that require human intervention and expertise. This not only improves the overall productivity of the company but also allows for a more agile response to market demands and customer needs.

## Requirements :

### 1. Salesforce CRM Implementation

- Store and manage vehicle details, stock availability, and dealer information in Salesforce.
- Track customer orders, test drives, and service requests efficiently.
- Automate workflows to assign orders to the nearest dealer based on customer location.

## **2. Process Automation**

- Prevent order placement if the vehicle is out of stock.
- Auto-assign orders to the nearest dealer based on the customer's location.
- Send automated email reminders for scheduled test drives.

## **3. Apex and Trigger**

- Implement Apex triggers to enforce business rules such as stock validation and automatic dealer assignment.
- Use trigger handlers to follow best practices and ensure modularity and maintainability.

## **4. Batch Jobs**

- Develop a batch Apex job to periodically check vehicle stock levels and update availability.
- Send scheduled email notifications for stock replenishment and order processing.

## **What you'll learn :**

1. Data Modelling
2. Fields and Relationships
3. Lightning App Builder
4. Record Triggered Flows
5. Apex and Apex Triggers
6. Batch Apex
7. Scheduled Apex

# MileStone 1 :

## Activity 1: Creating Developer Account

Creating a developer org in salesforce.

1. Go to <https://developer.salesforce.com/signup>
2. On the sign up form, enter the following details :
  - First name & Last name
  - Email
  - Role : Developer
  - Company : College Name
  - Country : India
  - Postal Code : pin code
  - Username : should be a combination of your name and company

This need not be an actual email id, you can give anything in the format : username@organization.com. Click on **Sign me up** after filling these.

## Activity 2: Account Activation

1. Go to the inbox of the email that you used while signing up. Click on **Verify Account** to activate your account. The email may take 5-10 mins.
2. Give a password and answer a security question and click on **Change Password**.
3. You will then be redirected to your Salesforce setup page.

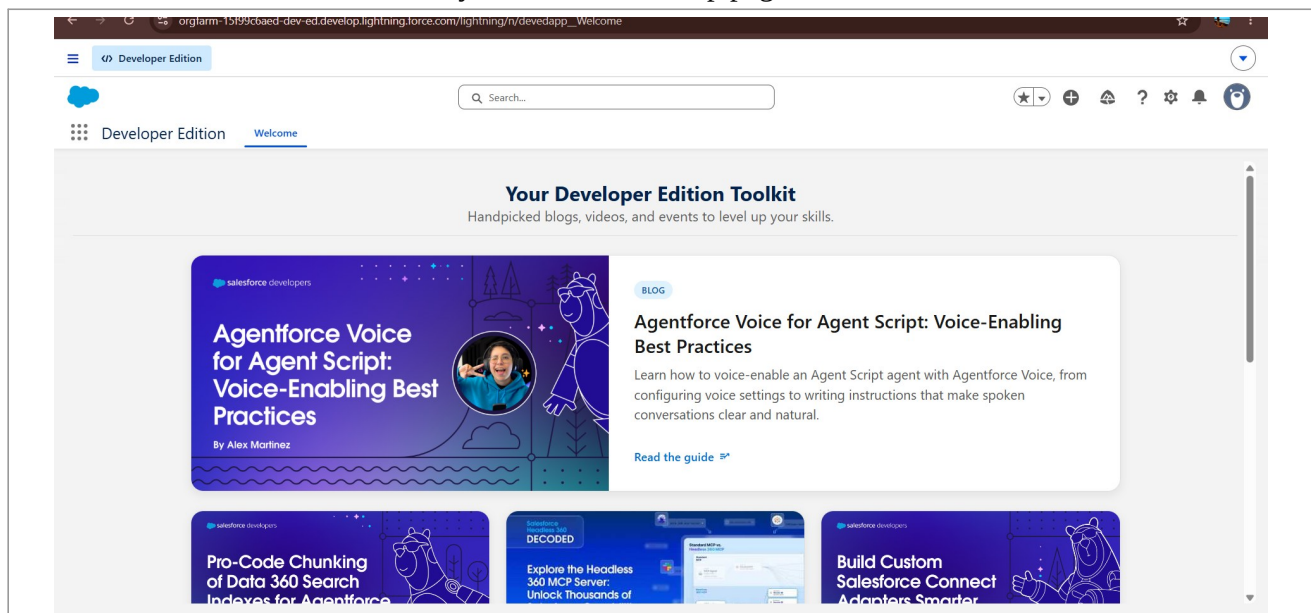


Figure 1: Developer Edition org is ready and opens on the Developer Edition welcome page.

# MileStone 2 :

## Objects & Relationships

| Object Name                       | Purpose                           | Relationships                   |
|-----------------------------------|-----------------------------------|---------------------------------|
| <b>Vehicle__c</b>                 | Stores vehicle details            | Related to Dealer & Orders      |
| <b>Vehicle_Dealer__c</b>          | Stores authorized dealer info     | Related to Orders               |
| <b>Vehicle_Customer__c</b>        | Stores customer details           | Related to Orders & Test Drives |
| <b>Vehicle_Order__c</b>           | Tracks vehicle purchases          | Related to Customer & Vehicle   |
| <b>Vehicle_Test_Drive__c</b>      | Tracks test drive bookings        | Related to Customer & Vehicle   |
| <b>Vehicle_Service_Request__c</b> | Tracks vehicle servicing requests | Related to Customer & Vehicle   |

### To Navigate to Setup page:

Click on gear icon → click Setup.

### Activity 1: Create Vehicle Object

The purpose of creating a Vehicle custom object is to have a clear picture of the vehicle details.

To create an object:

1. From the setup page → click on Object Manager → click on Create → click on Custom Object.
2. Enter the label name → **Vehicle**; Plural label name → **Vehicles**.
3. Enter Record Name Label and Format: Record Name → **Vehicle Name**, Data Type → **Text**.
4. Click on **Allow reports**, **Allow search** → **Save**.

### Activity 2: Create Dealer Object

The purpose of creating a Dealer object is to have detailed information about the dealer.

1. From the setup page → click on Object Manager → click on Create → click on Custom Object.
2. Enter the label name → **Vehicle Dealer**; Plural label name → **Vehicle Dealers**.
3. Record Name → **Dealer Name**, Data Type → **Text**.
4. Click on **Allow reports**, **Allow search** → **Save**.

### Activity 3: Creating remaining objects

Repeat the same steps to create Vehicle Customer, Vehicle Order, Vehicle Test Drive and Vehicle Service Request.

# Milestone 3 - Tabs

## Activity 1: Creating a Custom Tab (Vehicle)

1. Go to setup page → type **Tabs** in Quick Find bar → click on Tabs → **New** (under Custom Object Tabs).
2. Select Object (Vehicle) → select any tab style → Next (Add to profiles page) keep it as default → Next (Add to Custom App) keep it as default → Save.

## Activity 2:

Now create tabs for the other objects as well, which we have created above.

# Milestone 4 - The Lightning App :

## Activity 1: Create a Lightning App

To create a Lightning app page:

1. Go to setup page → search “app manager” in Quick Find → select **App Manager** → click on **New Lightning App**.
2. Fill the app details and branding: App Name : **WhatNext Vision Motors**; Developer Name : auto populated; Description : give a meaningful description; Image : optional; Primary color hex value : keep default.
3. Click Next → (App Options page) keep as default → Next → (Utility Items) keep as default → Next.

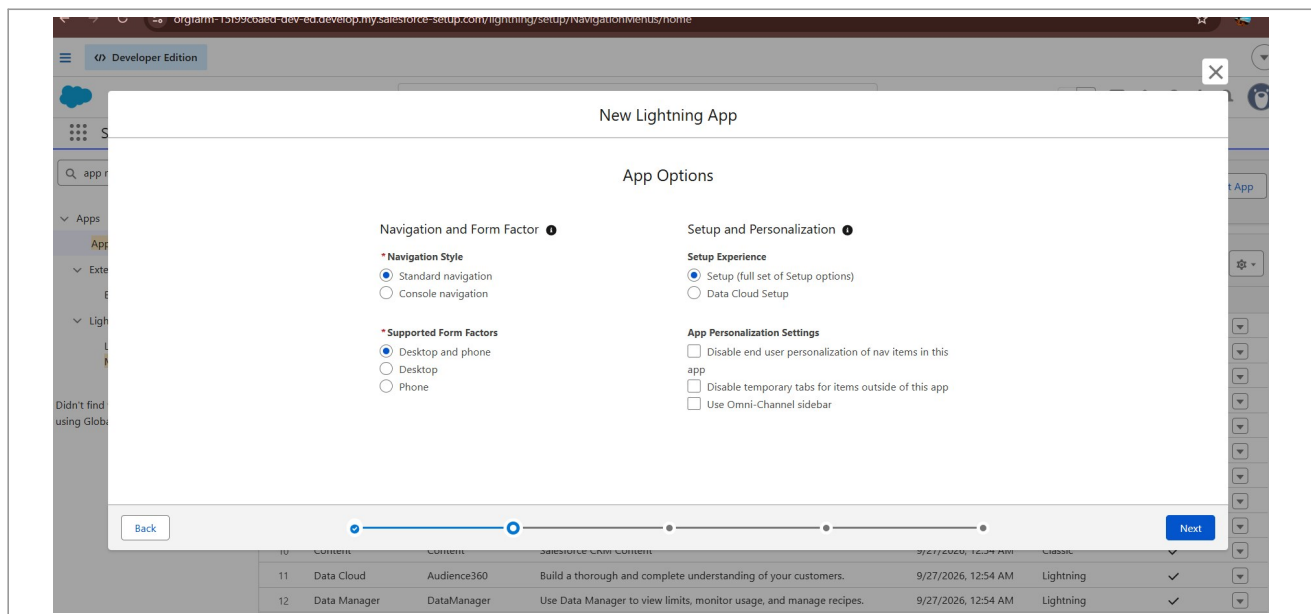


Figure 2: New Lightning App – App Options: Standard navigation, Desktop and phone, Setup experience left as default.

1. **Add Navigation Items:** search the items (Vehicle, Dealer, Customer, Order, Test Drive, Service Request, Reports, Dashboard) in the search bar and move them using the arrow button → Next. Select the custom objects created in the previous activity.
2. **Add User Profiles:** search profile (System Administrator) → click the arrow button → **Save & Finish**.

# Milestone 5 : Fields & Relationships

## Key Fields for Each Object

### 1. Vehicle\_\_c (Custom Object)

- Vehicle Name (Text)
- Vehicle\_Model\_\_c (Picklist: Sedan, SUV, EV, etc.)
- Stock\_Quantity\_\_c (Number)
- Price\_\_c (Currency)
- Dealer\_\_c (Lookup to Vehicle\_Dealer\_\_c)
- Status\_\_c (Picklist: Available, Out of Stock, Discontinued)

### 2. Vehicle\_Dealer\_\_c (Custom Object)

- Dealer\_Name\_\_c (Text)
- Dealer\_Location\_\_c (Text)
- Dealer\_Code\_\_c (Auto Number)
- Phone\_\_c (Phone)
- Email\_\_c (Email)

### 3. Vehicle\_Order\_\_c (Custom Object)

- Vehicle\_Customer\_\_c (Lookup to Vehicle\_Customer\_\_c)
- Vehicle\_\_c (Lookup to Vehicle\_\_c)
- Order\_Date\_\_c (Date)
- Status\_\_c (Picklist: Pending, Confirmed, Delivered, Canceled)
- Dealer\_\_c (Lookup to Vehicle\_Dealer\_\_c) – needed by Flow 1 to store the nearest dealer

### 4. Vehicle\_Customer\_\_c (Custom Object)

- Customer\_Name\_\_c (Text)
- Email\_\_c (Email)
- Phone\_\_c (Phone)
- Address\_\_c (Text)
- Preferred\_Vehicle\_Type\_\_c (Picklist: Sedan, SUV, EV, etc.)

### 5. Vehicle\_Test\_Drive\_\_c (Custom Object)

- Vehicle\_Customer\_\_c (Lookup to Vehicle\_Customer\_\_c)
- Vehicle\_\_c (Lookup to Vehicle\_\_c)
- Test\_Drive\_Date\_\_c (Date)
- Status\_\_c (Picklist: Scheduled, Completed, Canceled)

### 6. Vehicle\_Service\_Request\_\_c (Custom Object)

- Vehicle\_Customer\_\_c (Lookup to Vehicle\_Customer\_\_c)

- Vehicle\_\_c (Lookup to Vehicle\_\_c)
- Service\_Date\_\_c (Date)
- Issue\_Description\_\_c (Text)
- Status\_\_c (Picklist: Requested, In Progress, Completed)

### Activity 1: Creating a Field in Vehicle Object

1. Go to setup → Object Manager → type object name (Vehicle) in Quick Find → click on the object.
2. Click **Fields & Relationships** → **New**.
3. Select Data type as **Picklist** → Next.
4. Field Label: **Vehicle Model**; enter the picklist values (Sedan, SUV, EV) → Next → Next → Save and New.

### Activity 2: Creating Stock Quantity field in Vehicle object

1. Repeat steps 1 & 2 from Activity 1.
2. Select Data type as **Number** → Next.
3. Field Label: **Stock Quantity** → Next → Next → Save and New.

### Activity 3: Creating a Lookup Relationship in Vehicle Object to Dealer Object

1. Select Data Type as **Lookup Relationship** → Next.
2. Select the Vehicle Dealer object in the Related To field → Next.
3. Field Label / Field Name: **Dealer** → Next → Next and Save.

### Activity 4: Creating remaining objects and fields

Create the remaining fields listed above for each object in the same way.

# MileStone 6 :

## Flow 1 : Creating a record triggered flow to assign nearest dealer to the customer's location

**Step 1 :** In Quick Find, type **Flows** and click on Flows. Click **New Flow**.

**Step 2 :** Select **Triggered Automations**, then select **Record-Triggered Flow**.

**Step 3 :** Select the **Vehicle Order** object. Trigger the flow when: **A record is created**. Optimize the flow for **Actions and Related Records**.

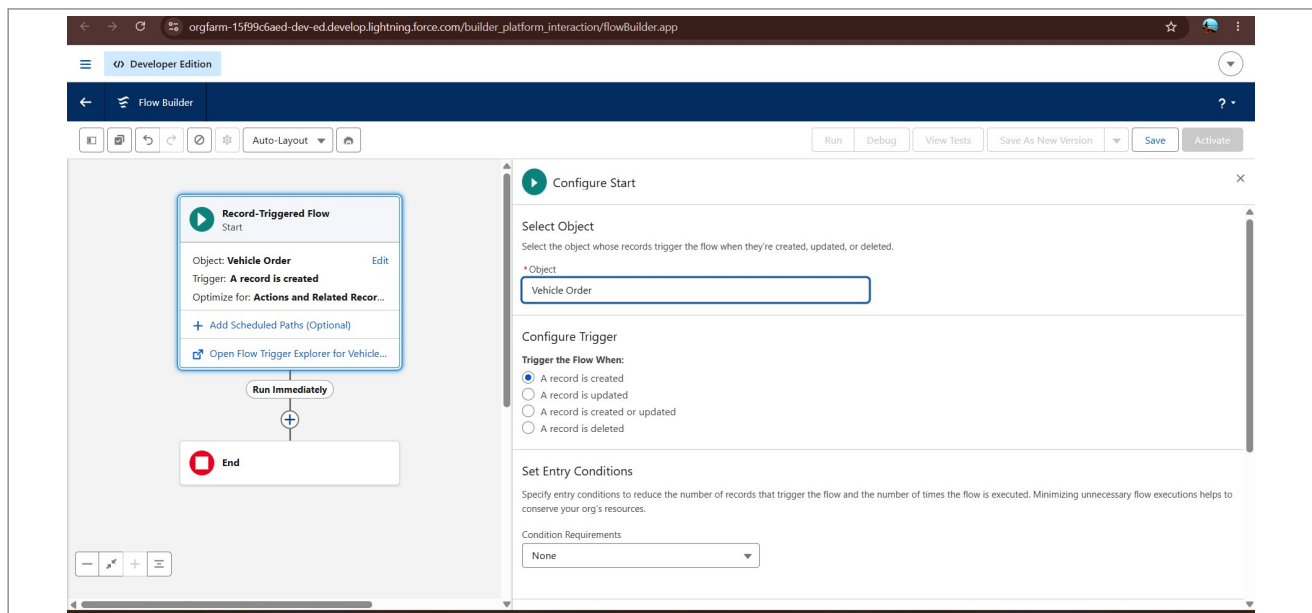


Figure 3: Configure Start: Object = Vehicle Order, trigger = A record is created.

**Step 4 :** Set Entry Conditions – All Conditions Are Met (AND): Field : Status\_\_c, Operator : Equals, Value : Pending.

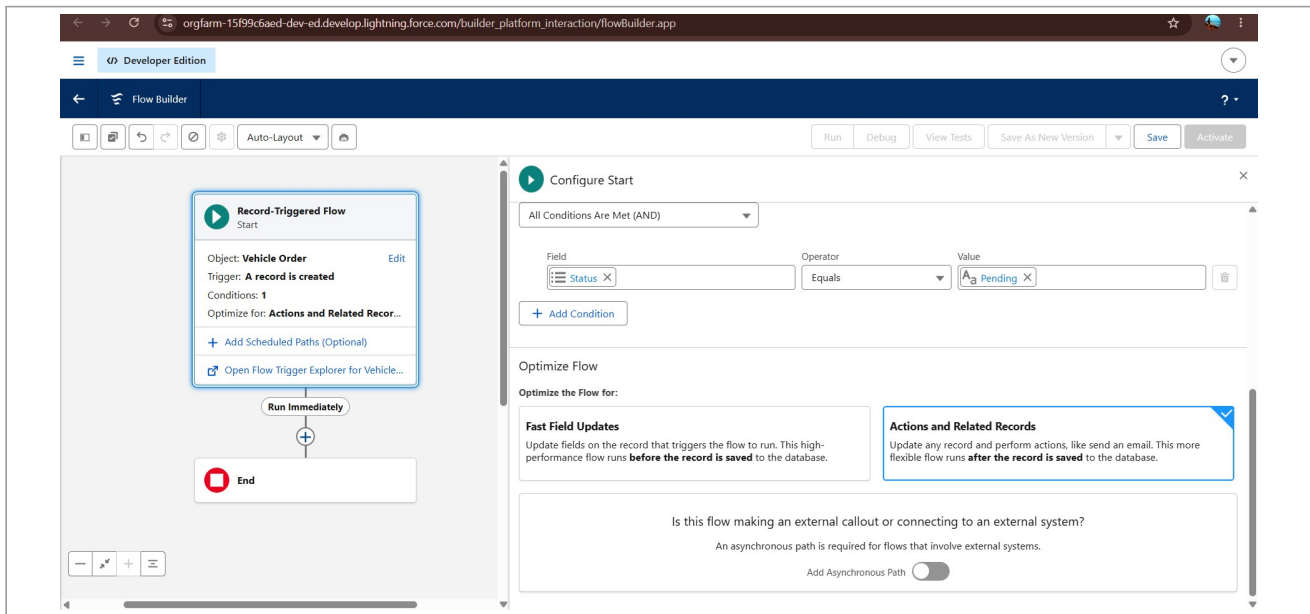


Figure 4: Entry condition Status Equals Pending, optimized for Actions and Related Records.

**Step 5 :** Click + → select **Get Records**. Label : **Get Customer**; Object : Vehicle Customer; Condition : Record ID Equals the Vehicle Customer on the triggering Vehicle Order record (`{!$Record.Vehicle_Customer__c}`); How many records to store : Only the first record; How to store record data : Automatically store all fields.

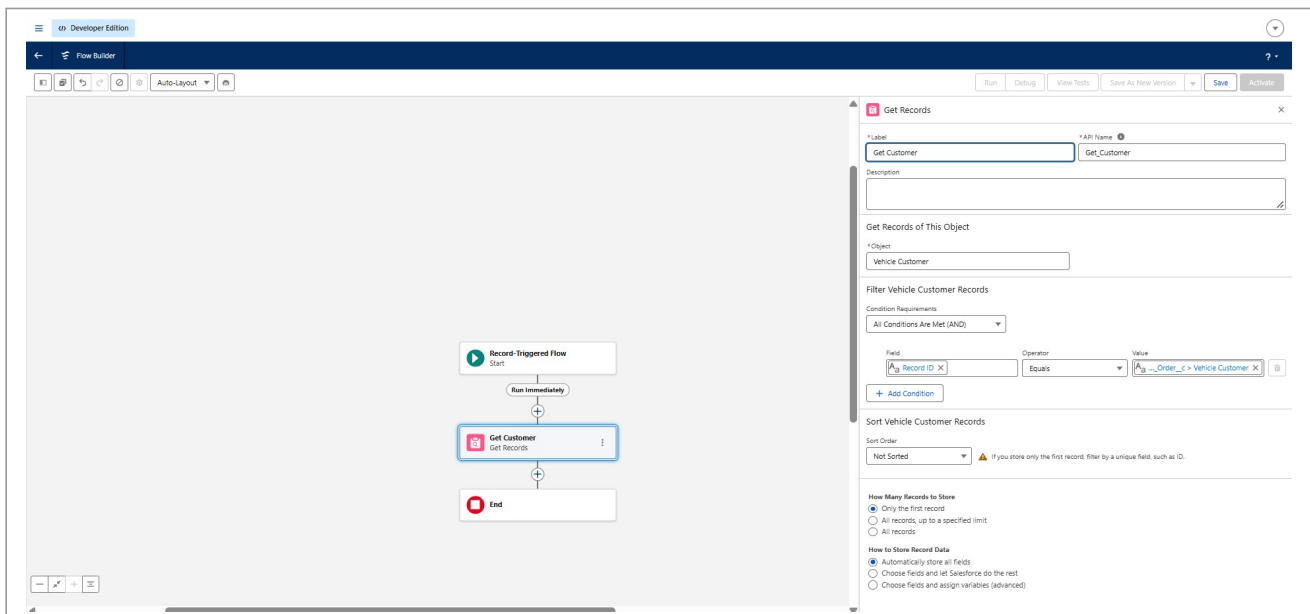


Figure 5: Get Records element – Get Customer.

**Step 6 :** Click + → select **Get Records**. Label : **Get Nearest Dealer**; Object : Vehicle Dealer; Condition : Dealer\_Location\_\_c Equals the customer's address (`{!Get_Customer.Address__c}`); Only the first record; Automatically store all fields.

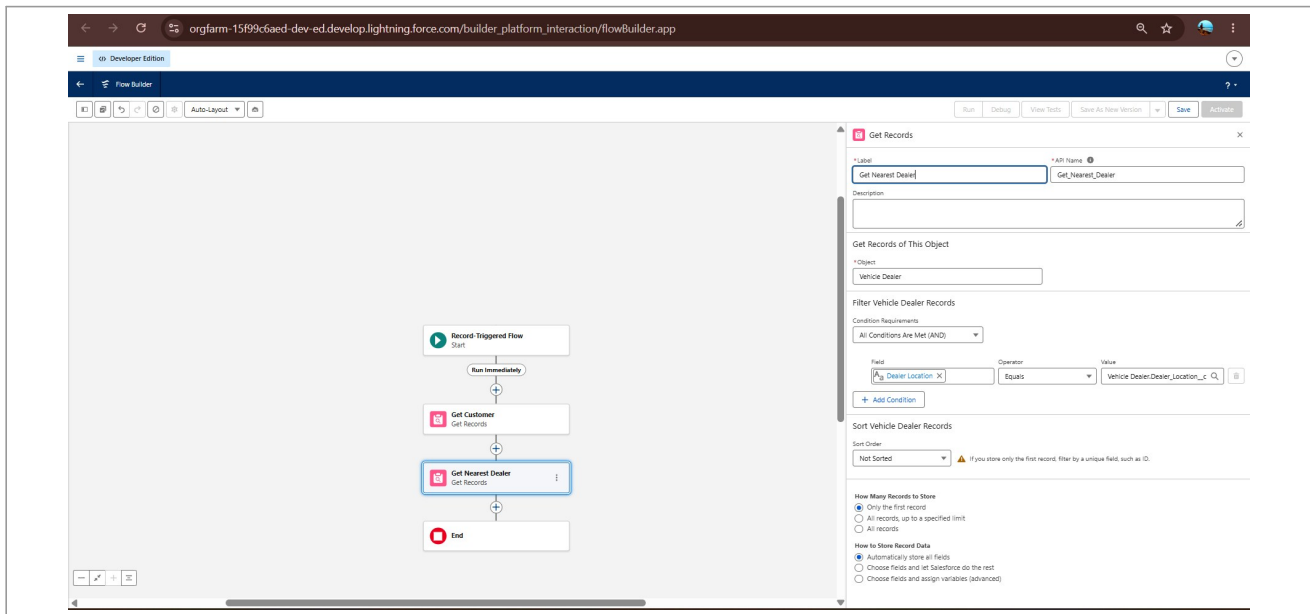


Figure 6: Get Records element – Get Nearest Dealer.

**Step 7 :** Click + → select **Update Records**. Label : **Update Vehicle Order**; How to find records to update : **Use the vehicle order record that triggered the flow.**

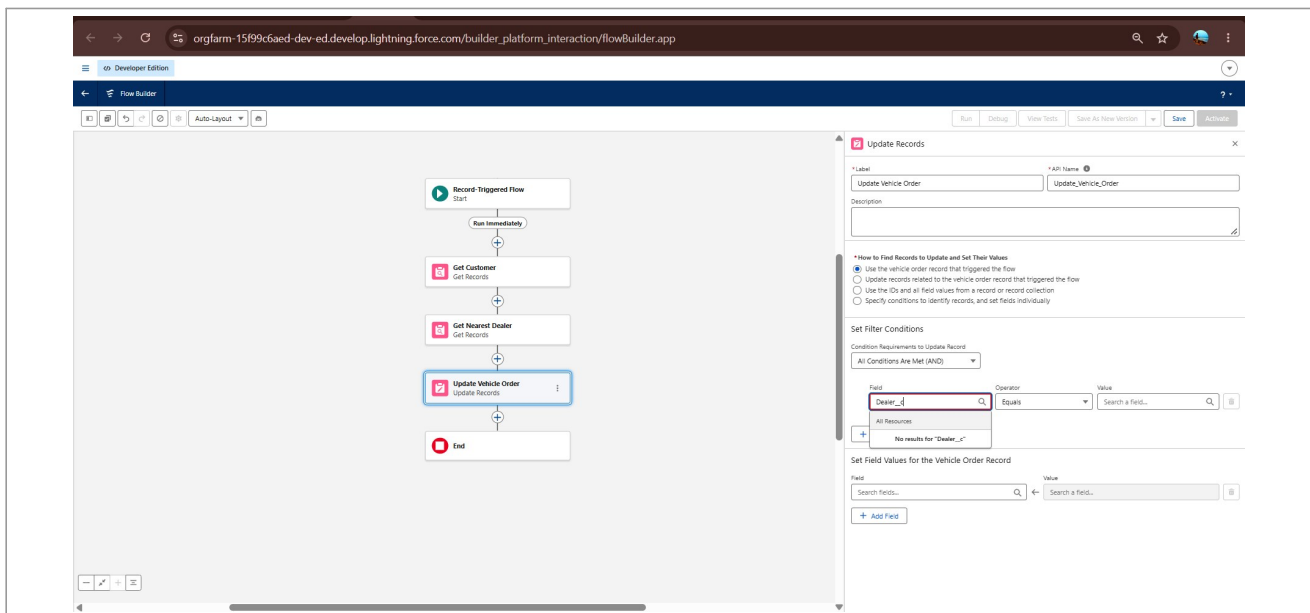


Figure 7: Update Records element added after Get Nearest Dealer.

**Step 8 :** Set Filter Conditions : **None—Always Update Record.**

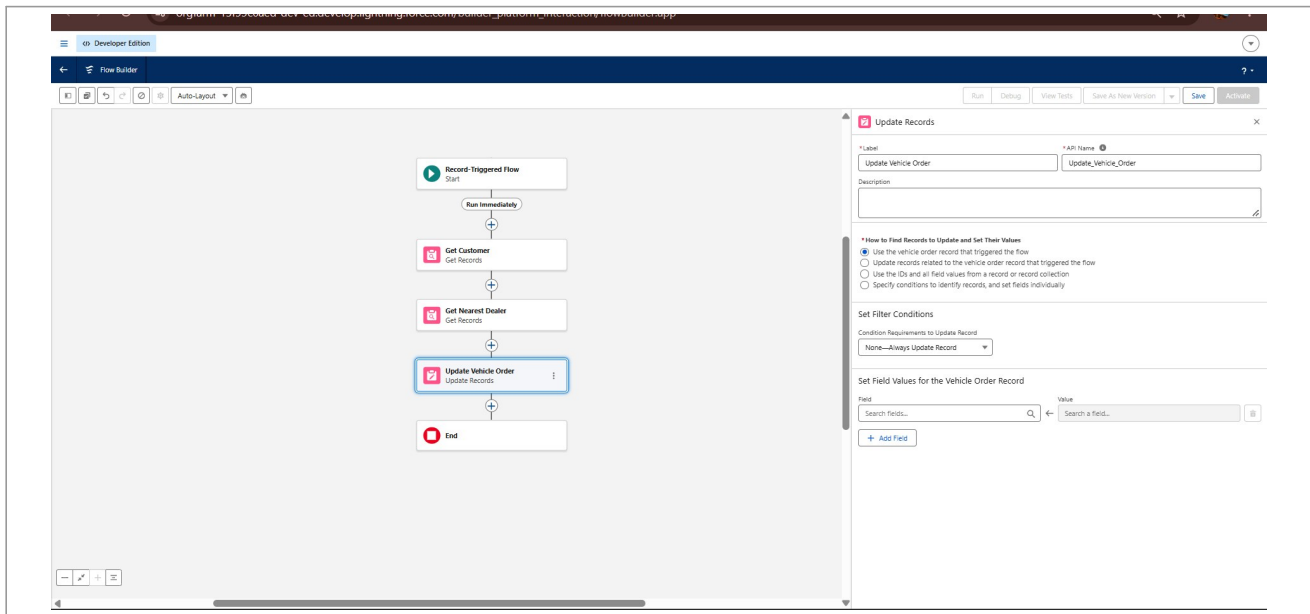


Figure 8: Update Vehicle Order – condition requirement set to None, Always Update Record.

**Step 9 :** Under *Set Field Values for the Vehicle Order Record*, choose the Dealer lookup field and set its value to the nearest dealer's Id ({!Get\_Nearest\_Dealer.Id}).

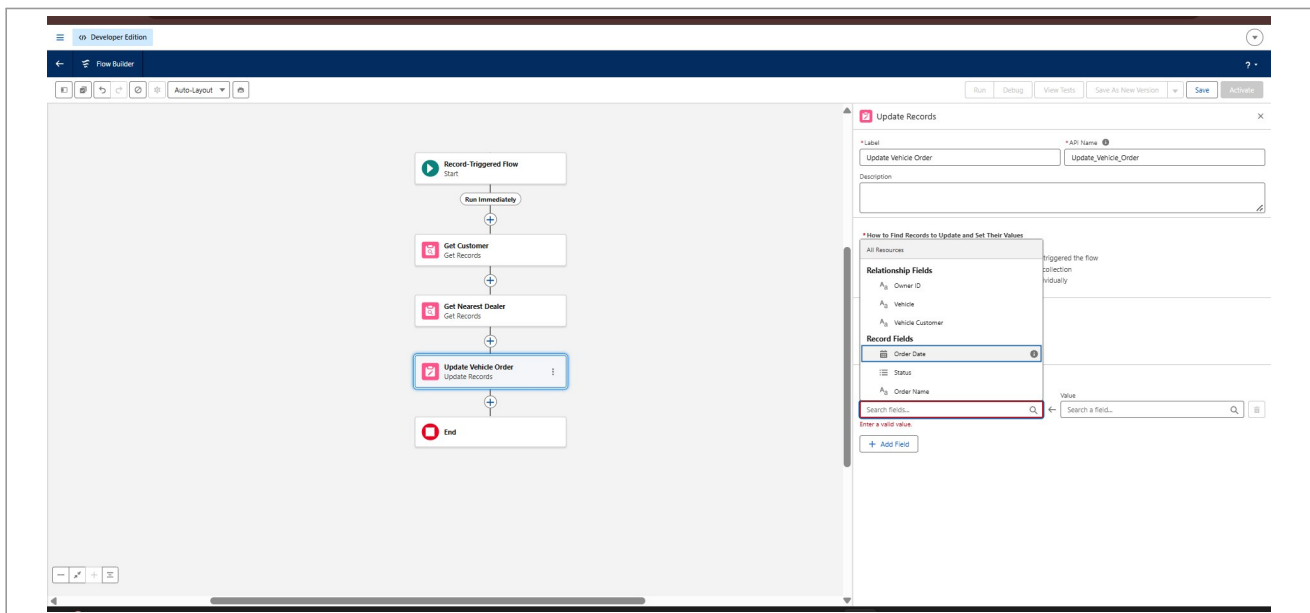


Figure 9: Field picker for the Vehicle Order record (Owner ID, Vehicle, Vehicle Customer, Order Date, Status, Order Name).

**Note:** the field list above does not yet contain a Dealer field. Before Step 9, create a lookup on Vehicle Order: Object Manager → Vehicle Order → Fields & Relationships → New → Lookup Relationship → Related To **Vehicle Dealer** → Field Label **Dealer**. Then reopen the flow and it will appear in the picker.

**Step 10 :** Click **Save**, give the Flow Label **Auto Assign Dealer**, then click **Activate**.

## Flow 2 : Creating record triggered flow to send an email to the customer reminding about the test drive

**Step 1 :** Select **Record-Triggered Flow**.

**Step 2 :** Select the **Vehicle Test Drive** object. Trigger the flow when : A record is created or updated. Entry condition (All Conditions Are Met): Status\_\_c Equals Scheduled.

**Step 3 :** Click + **Add Scheduled Paths** below the trigger. Label : Reminder Before Test Drive; Time Source : Test\_Drive\_Date\_\_c; Offset Number : 1; Offset Options : Days Before; click Done.

**Step 4 :** On the scheduled path click + **Add Element** → **Get Records**. Label : Get Customer Information; Object : Vehicle\_Customer\_\_c; Filter : Id = {!\$Record.Vehicle\_Customer\_\_c}; Only the first record; Automatically store all fields.

**Step 5 :** Send Reminder Email – click + **Add Element** → **Action**.

1. Action Type : Send Email
2. Label : Send Test Drive Reminder
3. Subject : “Reminder: Your Test Drive is Tomorrow!”
4. Recipient Address List : {!Get\_Customer\_Information.Email\_\_c}
5. Rich-Text-Formatted Body : True
6. Body : create a Text Template resource with API Name **EmailSent** and the text below, then select it as the Body.

Dear {!Get\_Customer\_Information.Name},

This is a reminder that your test drive for {!\$Record.Vehicle\_\_c} is scheduled for tomorrow. If you need to reschedule, contact us at [Support Email].  
Thank you, WhatNext Vision Motors Team

**Step 6 :** Click **Save**, Label Name : **Test Drive Reminder**.

**Step 7 :** Click **Activate**, then test the flow by creating a Test Drive record with Status = Scheduled and checking the customer’s inbox.

## MileStone 7 :

### Create Apex and Trigger Batch Jobs :

**Step 1 :** Click **Developer Console** from the Gear icon.

**Step 2 :** Click File → New → Apex Class.

**Step 3 :** Give the Apex class a name. Class Name : **VehicleOrderTriggerHandler**

**Step 4 :** Write the Apex code.

## Source Code :

```
public class VehicleOrderTriggerHandler {

    public static void handleTrigger(List<Vehicle_Order__c> newOrders,
        Map<Id, Vehicle_Order__c> oldOrders, Boolean isBefore, Boolean isAfter,
        Boolean isInsert, Boolean isUpdate) {
        if (isBefore) {
            if (isInsert || isUpdate) {
                preventOrderIfOutOfStock(newOrders);
            }
        }

        if (isAfter) {
            if (isInsert || isUpdate) {
                updateStockOnOrderPlacement(newOrders);
            }
        }
    }

    // Method to prevent orders when the vehicle is out of stock
    private static void preventOrderIfOutOfStock(List<Vehicle_Order__c> orders) {
        Set<Id> vehicleIds = new Set<Id>();
        for (Vehicle_Order__c order : orders) {
            if (order.Vehicle__c != null) {
                vehicleIds.add(order.Vehicle__c);
            }
        }

        if (!vehicleIds.isEmpty()) {
            Map<Id, Vehicle__c> vehicleStockMap = new Map<Id, Vehicle__c>();
            for (Vehicle__c vehicle : [SELECT Id, Stock_Quantity__c
                FROM Vehicle__c WHERE Id IN :vehicleIds]) {
                vehicleStockMap.put(vehicle.Id, vehicle);
            }

            for (Vehicle_Order__c order : orders) {
                if (vehicleStockMap.containsKey(order.Vehicle__c)) {
                    Vehicle__c vehicle = vehicleStockMap.get(order.Vehicle__c);
                    if (vehicle.Stock_Quantity__c <= 0) {
                        order.addError('This vehicle is out of stock. Order cannot be placed.');
                    }
                }
            }
        }
    }

    // Method to update vehicle stock when an order is placed
    private static void updateStockOnOrderPlacement(List<Vehicle_Order__c> orders) {
        Set<Id> vehicleIds = new Set<Id>();
        for (Vehicle_Order__c order : orders) {
            if (order.Vehicle__c != null && order.Status__c == 'Confirmed') {
                vehicleIds.add(order.Vehicle__c);
            }
        }

        if (!vehicleIds.isEmpty()) {
            Map<Id, Vehicle__c> vehicleStockMap = new Map<Id, Vehicle__c>();
            for (Vehicle__c vehicle : [SELECT Id, Stock_Quantity__c
                FROM Vehicle__c WHERE Id IN :vehicleIds]) {
                vehicleStockMap.put(vehicle.Id, vehicle);
            }

            List<Vehicle__c> vehiclesToUpdate = new List<Vehicle__c>();
            for (Vehicle_Order__c order : orders) {
                if (vehicleStockMap.containsKey(order.Vehicle__c)) {
```

```

        Vehicle__c vehicle = vehicleStockMap.get(order.Vehicle__c);
        if (vehicle.Stock_Quantity__c > 0) {
            vehicle.Stock_Quantity__c -= 1;
            vehiclesToUpdate.add(vehicle);
        }
    }
}

if (!vehiclesToUpdate.isEmpty()) {
    update vehiclesToUpdate;
}
}
}
}

```

**Step 5 :** Write the Trigger – File → New → Apex Trigger.

**Step 6 :** Trigger Name : **VehicleOrderTrigger**; sObject : **Vehicle\_Order\_\_c** → Submit.

**Step 7 :** Call the Apex handler class from the trigger.

```

trigger VehicleOrderTrigger on Vehicle_Order__c (before insert, before update,
                                                after insert, after update) {
    VehicleOrderTriggerHandler.handleTrigger(trigger.new, trigger.oldMap,
        trigger.isBefore, trigger.isAfter, trigger.isInsert, trigger.isUpdate);
}

```

**Step 8 :** Create the Batch Job. Scenario: a customer places an order, but the vehicle is out of stock.

- The order remains pending.
- After new stock is added, the batch job updates the order to confirmed.

Create a new Apex class named **VehicleOrderBatch**:

```

global class VehicleOrderBatch implements Database.Batchable<sObject> {

    global Database.QueryLocator start(Database.BatchableContext bc) {
        return Database.getQueryLocator([
            SELECT Id, Status__c, Vehicle__c
            FROM Vehicle_Order__c
            WHERE Status__c = 'Pending'
        ]);
    }

    global void execute(Database.BatchableContext bc, List<Vehicle_Order__c> orderList) {
        Set<Id> vehicleIds = new Set<Id>();

        for (Vehicle_Order__c order : orderList) {
            if (order.Vehicle__c != null) {
                vehicleIds.add(order.Vehicle__c);
            }
        }

        if (!vehicleIds.isEmpty()) {
            Map<Id, Vehicle__c> vehicleStockMap = new Map<Id, Vehicle__c>();
            for (Vehicle__c vehicle : [SELECT Id, Stock_Quantity__c
                                      FROM Vehicle__c WHERE Id IN :vehicleIds]) {
                vehicleStockMap.put(vehicle.Id, vehicle);
            }

            List<Vehicle_Order__c> ordersToUpdate = new List<Vehicle_Order__c>();
            List<Vehicle__c> vehiclesToUpdate = new List<Vehicle__c>();

            for (Vehicle_Order__c order : orderList) {
                if (vehicleStockMap.containsKey(order.Vehicle__c)) {

```

```

        Vehicle__c vehicle = vehicleStockMap.get(order.Vehicle__c);
        if (vehicle.Stock_Quantity__c > 0) {
            order.Status__c = 'Confirmed';
            vehicle.Stock_Quantity__c -= 1;
            ordersToUpdate.add(order);
            vehiclesToUpdate.add(vehicle);
        }
    }
}

if (!ordersToUpdate.isEmpty()) {
    update ordersToUpdate;
}

if (!vehiclesToUpdate.isEmpty()) {
    update vehiclesToUpdate;
}
}

global void finish(Database.BatchableContext bc) {
    System.debug('Vehicle order batch job completed.');
```

**Step 9 :** Create the Schedule class (**VehicleOrderBatchScheduler**) and invoke the Batch class from it.

```

global class VehicleOrderBatchScheduler implements Schedulable {
    global void execute(SchedulableContext sc) {
        VehicleOrderBatch batchJob = new VehicleOrderBatch();
        Database.executeBatch(batchJob, 50); // 50 is the batch size
    }
}
```

**Step 10 :** Schedule the Batch Job to run every night at midnight. Open **Debug** → **Open Execute Anonymous Window**, paste the code below and click Execute.

```

String cronExp = '0 0 0 * * ?'; // Runs daily at 12:00 AM (midnight)
System.schedule('Daily Vehicle Order Processing', cronExp, new VehicleOrderBatchScheduler());
```

**Step 11 :** You can check where the scheduled job is running: Setup → Quick Find → **Scheduled Jobs**.

# Thank you